



Docker Learning Series - Day 4 (Part 1)

Docker Volumes & Data Persistence

(Handwritten Engineering Notes)



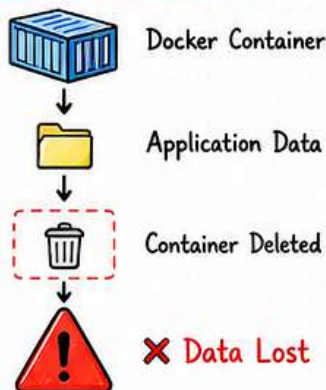
Docker Volumes • Data Persistence • Named Volumes • Anonymous Volumes
• Volume Lifecycle • Docker Storage

① Why Data Persistence?

- Docker containers are Ephemeral
- Container writable layer is temporary
- Data is lost when containers are deleted
- Production applications require persistent storage

★ Containers can be recreated anytime, but production data should never be lost.

Problem Architecture

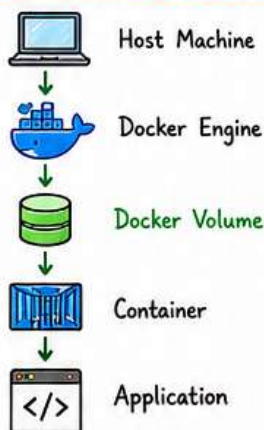


② What is Docker Volume?

- Docker-managed persistent storage
- Exists outside container lifecycle
- Independent from containers
- Can be attached to multiple containers
- Recommended for production workloads

★ A Docker Volume survives even when the container is deleted.

Docker Volume Architecture



③ Types of Docker Volumes

Named Volumes

- User-defined volume name
- Easy to identify
- Easy backup
- Easy restore
- Best choice for production

Example:

```
$ docker volume create mysql-data
```

Terminal Output:

```
mysql-data
```

Anonymous Volumes

- Docker automatically generates volume name
- Random ID
- Hard to manage
- Mostly used for temporary workloads

Example:

```
$ docker run -v /var/lib/mysql mysql
```

Example Output:

```
8d4ac7f85efc2c...
```

★ Named Volumes are recommended over Anonymous Volumes in production.

④ Docker Volume Lifecycle

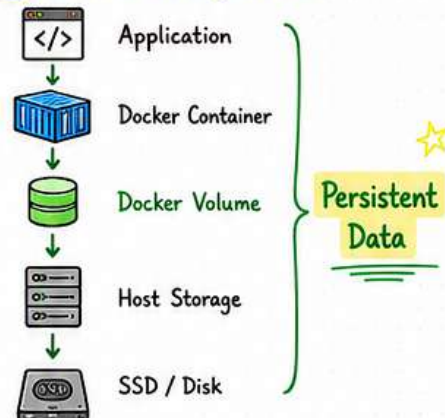
- + Create Volume
- ↓
- Attach to Container
- ↓
- Store Application Data
- ↓
- Stop Container
- ↓
- Delete Container
- ↓
- Volume Still Exists
- ↓
- Create New Container
- ↓
- Reuse Same Volume
- ↓
- ✓ Data Still Available

★
Persistent Storage

⑤ Docker Volume Commands

- | | |
|---|---|
| <pre>\$ docker volume create my-volume</pre> | Purpose:
Creates a new Docker Volume. |
| <pre>\$ docker volume ls</pre> | Purpose:
Lists all Docker Volumes. |
| <pre>\$ docker volume inspect my-volume</pre> | Purpose:
Displays detailed volume information. |
| <pre>\$ docker volume rm my-volume</pre> | Purpose:
Removes a Docker Volume. |
| <pre>\$ docker volume prune</pre> | Purpose:
Deletes all unused Docker Volumes. |

⑥ Docker Storage Flow



Production Tips

- ✓ Use Named Volumes for databases
- ✓ Never store databases inside container writable layer
- ✓ Backup Docker Volumes regularly
- ✓ Remove unused volumes
- ✓ Share volumes only when required
- ✓ Monitor storage usage



Interview Sticky Notes

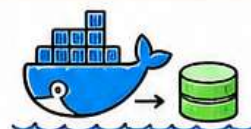
- ★ What is a Docker Volume?
- ★ Why are Docker Volumes required?
- ★ Difference between Docker Volume and Container Storage?
- ★ Difference between Named and Anonymous Volumes?
- ★ What happens to a Volume after deleting a Container?
- ★ Can multiple containers use the same Docker Volume?

Important Notes

✗	Container Storage →	Temporary
✓	Docker Volume →	Persistent
✗	Delete Container →	Container Removed
✓	Docker Volume →	Still Exists

Final Summary

Docker Volumes solve the biggest storage challenge in containerized environments by providing **persistent, reliable, and Docker-managed storage independent of container lifecycle**. Named Volumes are recommended for production workloads because they simplify **backup, restore, migration, and long-term data management**.





Docker Learning Series - Day 4 (Part 2)

Bind Mounts • tmpfs Mounts • Storage Comparison

Stay
Consistent
Keep
Learning!

Bind Mounts • tmpfs Mounts • Docker Volumes • Development vs Production • Storage Comparison

1 What is a Bind Mount?

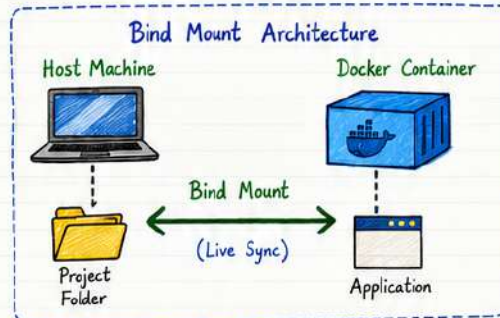
- Directly mounts a Host directory into a Container
- Host controls the files
- Live synchronization
- No Docker-managed storage
- Best suited for development

★ **Note:** Changes made on the Host are immediately reflected inside the Container.

Terminal Command `>_`

```
docker run -d \
-v $(pwd): /app \
nginx
```

Purpose:
Mount the current host directory into `/app` inside the container.



2 What is tmpfs Mount?

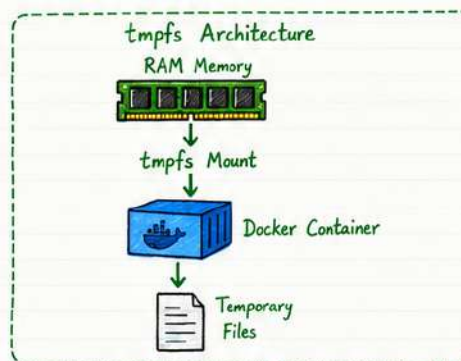
- Stores files only in RAM
- No disk usage
- Very high performance
- Automatically removed when the container stops
- Used for temporary or sensitive data

★ **Note:**
tmpfs is memory-based storage and does not persist after the container exits.

Terminal Command `>_`

```
docker run -d \
--tmpfs /tmp \
nginx
```

Purpose:
Create a temporary RAM-based filesystem inside the container.



3 Docker Storage Comparison

Feature	Docker Volume (Recommended)	Bind Mount (Development)	tmpfs (In-Memory)
Managed by Docker	✓	✗	✗
Persistent Storage	✓	✓	✗
Stored in RAM	✗	✗	✓
Host File Access	✗	✓	✗
Easy Backup	✓	Manual	✗
Best Performance	★★★★	★★★★	★★★★★
Development	★★★	★★★★★	★★
Production	★★★★★	★★	★★★

🏆 **Production Recommendation:** Use Docker Volume for production workloads!

4 Real World Use Cases

Docker Volume

- ✓ MySQL
- ✓ PostgreSQL
- ✓ MongoDB
- ✓ Redis
- ✓ Jenkins Home
- ✓ Prometheus Data

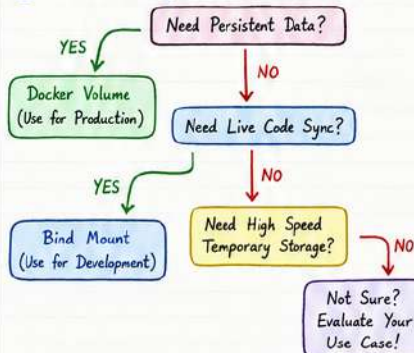
Bind Mount

- ✓ React
- ✓ Angular
- ✓ Node.js
- ✓ Python
- ✓ Java Development
- ✓ Live Code Editing

tmpfs

- ✓ Cache
- ✓ Session Data
- ✓ Secrets
- ✓ Temporary Files
- ✓ Build Cache
- ✓ High-Speed Processing

5 Which Storage Should You Choose?



6 Production Tips

- ✓ Use Docker Volumes for databases.
- ✓ Use Bind Mounts only during development.
- ✓ Avoid Bind Mounts for production database storage.
- ✓ Use tmpfs for temporary and sensitive files.
- ✓ Never store critical data inside tmpfs.
- ✓ Always choose storage based on application requirements.

Warning Notes

✗ **Bind Mount**
Host files can be accidentally modified.

✗ **tmpfs**
All data is lost after the container stops.

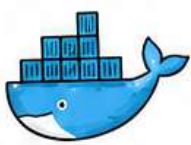
✓ **Docker Volume**
Safest option for persistent production data.

Final Summary ★

- Docker provides three different storage mechanisms.
- Docker Volumes are the recommended choice for production workloads requiring persistent data.
- Bind Mounts are ideal for development because they enable live synchronization between the Host and the Container.
- tmpfs provides high-speed in-memory storage for temporary files and sensitive workloads but does not persist after the container exits.
- Understanding the strengths and limitations of each storage option helps DevOps engineers choose the right solution for different application requirements.

Interview Sticky Notes

- What is a Bind Mount?
- Difference between Docker Volume and Bind Mount?
- What is tmpfs?
- Why is tmpfs faster?
- Which storage option is recommended for databases?
- Can Bind Mounts be used in production?
- Which storage type provides persistence?
- Which storage type stores data in RAM?



Docker Learning Series - Day 4 (Part 3)

Backup • Restore • Production Best Practices

(Handwritten Engineering Notes)

- ★ Today's Focus
- ✓ Backup & Restore
 - ✓ Disaster Recovery
 - ✓ Production Storage
 - ✓ Best Practices
 - ✓ Interview Questions

① Backup Docker Volumes

- Docker Volumes should be backed up regularly.
- Backup protects application and database data.
- Backup files can be restored anytime.

💡 Containers can be recreated, but production data must always be backed up.

Terminal Command

```
docker run --rm \
-v my-volume:/source \
-v $(pwd):/backup \
ubuntu \
tar czf /backup/backup.tar.gz /source
```

Purpose:
Create a compressed backup of the Docker Volume.

Backup Workflow



② Restore Docker Volume

- Restore previously backed up application data.
- Recover lost data after container failure.
- Restore into an existing or new Docker Volume.

💡 Backup is useful only when restore procedures are tested.

Terminal Command

```
docker run --rm \
-v my-volume:/restore \
-v $(pwd):/backup \
ubuntu \
tar xzf /backup/backup.tar.gz -C /
```

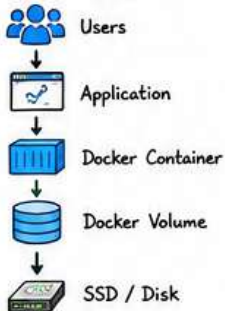
Purpose:
Restore archived data into the Docker Volume.

Restore Workflow



③ Production Database Workflow

Normal Workflow



Container Failure Recovery



✓ Zero Data Loss

★ This is why MySQL, PostgreSQL, MongoDB and Redis use Docker Volumes in production.



④ Disaster Recovery Workflow



Key Benefit
Even if server fails, your data is safe in the cloud. Restore and get back online quickly!



⑤ Production Best Practices

- ✓ Use Named Volumes.
- ✓ Schedule automatic backups.
- ✓ Verify restore procedures regularly.
- ✓ Store backups outside the Docker host.
- ✓ Encrypt backup archives.
- ✓ Monitor disk usage.
- ✓ Remove unused Docker Volumes.
- ✓ Document recovery procedures.
- ✓ Use Docker Volumes for production databases.
- ✓ Test disaster recovery plans periodically.

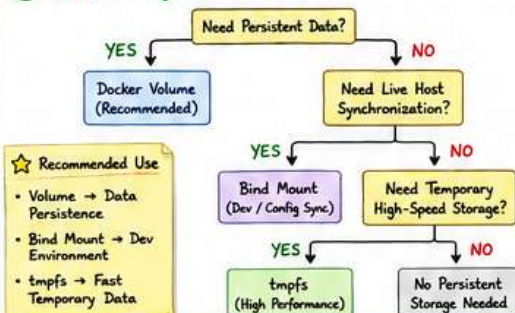
⑥ Common Production Mistakes

- ✗ Storing databases inside container writable layer.
- ✗ Never testing backup restoration.
- ✗ Keeping backups only on the same server.
- ✗ Deleting volumes accidentally.
- ✗ Using Bind Mounts for production databases.
- ✗ Ignoring storage capacity monitoring.

⑦ Useful Docker Commands

<code>docker volume ls</code>	List all volumes.
<code>docker volume inspect my-volume</code>	View detailed information.
<code>docker volume prune</code>	Remove unused volumes.
<code>docker system df</code>	Check Docker disk usage.
<code>docker volume rm my-volume</code>	Delete a Docker Volume.

⑧ Storage Decision Flow



Recommended Use

- Volume -> Data Persistence.
- Bind Mount -> Dev Environment.
- tmpfs -> Fast Temporary Data.

Interview Sticky Notes

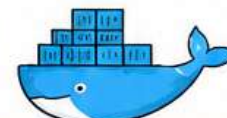
- ★ How do you back up a Docker Volume?
- ★ How do you restore a Docker Volume?
- ★ Why should backup files be stored outside the Docker host?
- ★ Why are Docker Volumes recommended for databases?
- ★ What happens if a container crashes?
- ★ Difference between Backup and Restore?
- ★ What are production storage best practices?
- ★ How do you prevent data loss in Docker?
- ★ Difference between Docker Volume and Bind Mount in production?
- ★ What is Disaster Recovery?

Warning Notes

- ✗ Containers are replaceable.
- ✓ Production data is not.
- ✗ Never depend on container writable storage.
- ✓ Always maintain verified backups.
- ✓ Regularly test restore procedures.

Final Summary

- Docker Volumes are the recommended storage solution for production workloads because they provide persistent, reliable, and Docker-managed storage independent of the container lifecycle. A strong backup strategy, tested restore process, and well-documented disaster recovery plan ensure business continuity even when containers or hosts fail. Every DevOps Engineer should know not only how to create Docker Volumes, but also how to protect, restore, and manage them in real production environments.



★ Remember
Backup Today
Restore Tomorrow
Protect Always
Stay Resilient! ❤️